

Fast KV Compaction via Attention Matching

Adam Zweiger¹ Xinghong Fu¹ Han Guo¹ Yoon Kim¹

<https://github.com/adamzweiger/compaction>

Abstract

Scaling language models to long contexts is often bottlenecked by the size of the key-value (KV) cache. In deployed settings, long contexts are typically managed through *compaction* in token space via summarization. However, summarization can be highly lossy, substantially harming downstream performance. Recent work on Cartridges (Eyuboglu et al., 2025) has shown that it is possible to *train* highly compact KV caches in latent space that closely match full-context performance, but at the cost of slow and expensive end-to-end optimization. This work describes an approach for *fast* context compaction in latent space through **Attention Matching**, which constructs compact keys and values to reproduce attention outputs and preserve attention mass at a per-KV-head level. We show that this formulation naturally decomposes into simple subproblems, some of which admit efficient closed-form solutions. Within this framework, we develop a family of methods that significantly push the Pareto frontier of compaction time versus quality, achieving up to 50× compaction in seconds on some datasets with little quality loss.

1. Introduction

Memory has emerged as a critical bottleneck in modern language models (LMs). As such systems are deployed in increasingly long-horizon settings—reasoning, multi-session dialogue, long-running agentic coding—the question of how to efficiently manage, compact, and retrieve contextual information has become paramount. The field is converging on a consensus: models that can remember more, and remember better, will unlock capabilities that remain out of reach for systems constrained by fixed context windows.

For autoregressive LMs based on the Transformer archi-

¹Massachusetts Institute of Technology. Correspondence to: Adam Zweiger <adamz@mit.edu>.

Preprint.

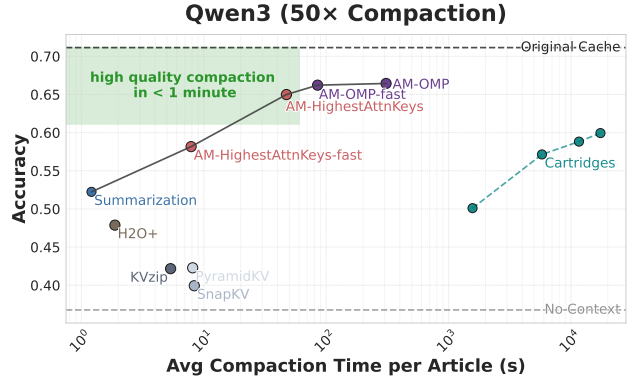


Figure 1. **Accuracy vs. Compaction Time Trade-off (Qwen3-4B; QuALITY)**. We compare downstream QA accuracy ($n = 894$) after compaction, plotted against the average wall-clock time required to compact a context (seconds, log-scale) using a single H100 GPU at a fixed 50× compaction ratio. Our attention-matching (AM) methods trace a speed-quality tradeoff and form the Pareto frontier, outperforming prior token-selection baselines and exceeding the performance of Cartridges (Eyuboglu et al., 2025) while being 2 orders of magnitude faster; additional Cartridges training may further improve its results.

ture, the memory bottleneck is specific: the key-value (KV) cache. Models must retain keys and values from all previous tokens, and in long-context settings, the KV cache can reach many gigabytes per request. Existing approaches to KV cache reduction, such as token eviction (Zhang et al., 2023; Li et al., 2024; Kim et al., 2025b), token merging (Wang et al., 2024; Zhang et al., 2024), and head sparsification (Xiao et al., 2024; 2025), degrade rapidly at high reduction ratios. As a result, real-world systems still rely heavily on summarizing or simply dropping older context (Anthropic, 2025; OpenAI, 2025). Effective *context compaction*¹—reducing KV cache size in a single pass while preserving downstream model behavior—remains an important open problem.

¹We use “compaction” (instead of “compression”) by analogy to log or memory compaction in systems, where many records are consolidated into a smaller representation that preserves the information needed for future access. The term “compaction” is similarly used by agentic coding tools (e.g., Claude Code, OpenAI Codex) to describe this operation, historically implemented purely via summarization.

Recently, Eyuboglu et al. (2025) propose Cartridges, an approach that can be interpreted as performing context compaction in *latent space*. Cartridges uses prefix-tuning (Li & Liang, 2021) on synthetic “self-study” data to *train* a compact KV cache for a given context. This enables compaction ratios as high as $50\times$ on long contexts with minimal performance loss. However, its end-to-end gradient-based optimization strategy can be prohibitively expensive, often requiring several GPU-hours to train a single Cartridge for a given context.

This paper describes a family of methods for fast KV compaction that can achieve Cartridges-level compaction ratios and quality while being orders of magnitude faster. Our approach is based on an **Attention Matching** objective: rather than training a compact KV cache end-to-end on output likelihoods, we directly optimize for compacted keys and values to reproduce the *attention outputs* and *attention mass* for every KV-head in every layer, matching these on a set of reference queries. We show that this Attention Matching objective decomposes into simple subroutines that admit closed-form solutions that are efficient to compute in practice, allowing us to avoid gradient descent entirely at compaction time. Different design choices within our framework yield a family of methods along the speed–performance frontier. Attention Matching enables compaction that is orders of magnitude faster than gradient-based optimization (e.g., minutes rather than hours), with little performance degradation at ratios up to $50\times$ (see Figure 1).

2. KV Compaction via Attention Matching

Consider the problem of compacting T tokens from a context. Let $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times d}$ denote the corresponding keys and values for a single KV-head, where d is the head dimension. Rotary embeddings are assumed to have already been applied to cached keys.

KV compaction aims to replace $(\mathbf{K}, \mathbf{V})$ with a shorter cache $(\mathbf{C}_k, \mathbf{C}_v) \in \mathbb{R}^{t \times d}$ with $t < T$ such that conditioning on $(\mathbf{C}_k, \mathbf{C}_v)$ behaves similarly to conditioning on $(\mathbf{K}, \mathbf{V})$ for any query $\mathbf{q} \in \mathbb{R}^{1 \times d}$.

A key requirement is compatibility with ordinary KV caching: compaction should remain valid even when the compacted prefix is concatenated with arbitrary uncompacted tokens (e.g., the most recent turn) or with future tokens appended after compaction (e.g., user queries or model continuations). Concretely, for any additional key–value blocks $(\mathbf{K}_{\text{fixed}}, \mathbf{V}_{\text{fixed}})$, we would ideally like

$$\text{Attn}\left(\mathbf{q}; \begin{bmatrix} \mathbf{K} \\ \mathbf{K}_{\text{fixed}} \end{bmatrix}, \begin{bmatrix} \mathbf{V} \\ \mathbf{V}_{\text{fixed}} \end{bmatrix}\right) \approx \text{Attn}\left(\mathbf{q}; \begin{bmatrix} \mathbf{C}_k \\ \mathbf{K}_{\text{fixed}} \end{bmatrix}, \begin{bmatrix} \mathbf{C}_v \\ \mathbf{V}_{\text{fixed}} \end{bmatrix}\right).$$

Written explicitly (omitting the $\sqrt{d}$ scaling for readability;

see Appendix A), we would like

$$\frac{\exp\left(\mathbf{q} \begin{bmatrix} \mathbf{K} \\ \mathbf{K}_{\text{fixed}} \end{bmatrix}^\top\right) \begin{bmatrix} \mathbf{V} \\ \mathbf{V}_{\text{fixed}} \end{bmatrix}}{\sum_{j=1}^{T+S} \exp\left(\mathbf{q} \begin{bmatrix} \mathbf{K} \\ \mathbf{K}_{\text{fixed}} \end{bmatrix}_j^\top\right)} \approx \frac{\exp\left(\mathbf{q} \begin{bmatrix} \mathbf{C}_k \\ \mathbf{K}_{\text{fixed}} \end{bmatrix}^\top\right) \begin{bmatrix} \mathbf{C}_v \\ \mathbf{V}_{\text{fixed}} \end{bmatrix}}{\sum_{j=1}^{t+S} \exp\left(\mathbf{q} \begin{bmatrix} \mathbf{C}_k \\ \mathbf{K}_{\text{fixed}} \end{bmatrix}_j^\top\right)}.$$

Directly enforcing this for all possible future $(\mathbf{K}_{\text{fixed}}, \mathbf{V}_{\text{fixed}})$ is challenging. Instead, we use the fact that attention over concatenated blocks decomposes into a mixture of each block’s *locally normalized* attention output, weighted by that block’s *attention mass*.² For a key block $\mathbf{K}$, define

$$\text{Mass}(\mathbf{q}; \mathbf{K}) = \sum_j \exp(\mathbf{q} \mathbf{K}_j^\top).$$

This suggests a sufficient compaction strategy that does not depend on unknown future tokens: over a set of reference (or “training”) queries, match (i) the compacted block’s local attention output and (ii) its attention mass.

One subtlety is that with $t < T$, exact mass matching using only $\mathbf{C}_k$ is impossible. For example, for $\mathbf{q} = \mathbf{0}$ we have $\text{Mass}(\mathbf{0}; \mathbf{K}) = T$, whereas $\text{Mass}(\mathbf{0}; \mathbf{C}_k) = t$ for any $\mathbf{C}_k \in \mathbb{R}^{t \times d}$. We therefore introduce a per-token scalar bias $\beta \in \mathbb{R}^t$, which multiplicatively reweights the contribution of each retained key to the mass. We later show that these scalar biases arise naturally in Attention Matching.

Specifically, we optimize $(\mathbf{C}_k, \beta, \mathbf{C}_v)$, where $\mathbf{C}_k, \mathbf{C}_v \in \mathbb{R}^{t \times d}$ and $\beta \in \mathbb{R}^t$ so that for all queries $\mathbf{q}$ of interest,

$$\frac{\exp(\mathbf{q} \mathbf{K}^\top) \mathbf{V}}{\sum_{j=1}^T \exp(\mathbf{q} \mathbf{K}_j^\top)} \approx \frac{\exp(\mathbf{q} \mathbf{C}_k^\top + \beta) \mathbf{C}_v}{\sum_{j=1}^t \exp(\mathbf{q} (\mathbf{C}_k)_j^\top + \beta_j)}, \quad (1)$$

$$\sum_{j=1}^T \exp(\mathbf{q} \mathbf{K}_j^\top) \approx \sum_{j=1}^t \exp(\mathbf{q} (\mathbf{C}_k)_j^\top + \beta_j). \quad (2)$$

Equation (1) matches the compacted block’s local attention output, while Eq. (2) matches its attention mass; together, these preserve the block’s contribution under concatenation with arbitrary fixed or future tokens (Appendix A.2). In contrast, methods that only drop or merge tokens without biases (e.g., evicting $T - t$ tokens) systematically underestimate the compacted block’s contribution during future decoding.

The scalar biases add negligible memory overhead (an additional factor of $\frac{2d+1}{2d}$) and negligible-to-zero change in attention runtime. They are supported in common attention implementations such as PyTorch SDPA and FlexAttention (Dong et al., 2025).

²Efficient attention implementations such as FlashAttention (Dao et al., 2022) and Cascade Inference (Ye et al., 2024; Juravsky et al., 2024) exploit the same decomposition.

Finally, although the compacted cache stores only t KV entries, it retains a *logical length* T so that newly appended tokens receive the same position IDs (and thus the same RoPE phases) as they would under the uncompact prefix. We view this as disentangling the cumulative length a cache has *seen* from its physical size.

3. Methods

We now present a family of methods for constructing (C_k, β, C_v) . Our approach is defined with respect to a set of *reference queries* $Q_{\text{ref}} = [q_1; \dots; q_n] \in \mathbb{R}^{n \times d}$, which serve as a proxy for the queries the model is likely to produce when attending to the context.

Jointly optimizing (C_k, β, C_v) is computationally difficult and typically requires gradient-based optimization, which we aim to avoid at compaction time. Instead, we first construct the compacted keys C_k , then compute the bias terms β , and finally obtain C_v , all in closed form.

3.1. Sampling Reference Queries Q_{ref}

We consider two main approaches for constructing Q_{ref} .

Repeat-prefill. Following KVzip (Kim et al., 2025b), we construct a sequence

“{ $\mathcal{C}$ } Repeat the previous context. { $\mathcal{C}$ }”

(with the model’s chat template applied appropriately), run a prefill pass on this sequence, and extract the query vectors used while the model reconstructs $\mathcal{C}$ (i.e., query activations starting from the instruction through the second $\mathcal{C}$). We found that the simpler variant of running a prefill on $\mathcal{C}$ alone, which we call *context-prefill* (and used by H2O (Zhang et al., 2023)), was cheaper (see Table 1) but performed slightly worse than *repeat-prefill*.

Self-study. Self-study (Eyuboglu et al., 2025) generates synthetic interactions conditioned on a fixed context $\mathcal{C}$. We use it as a lightweight way to broaden the query distribution: we prompt the model with $\mathcal{C}$ and a small set of fixed prompts (e.g., “Aggregate all key facts mentioned in the context”) or model-generated conversation-starters. We then sample responses and extract the resulting query vectors. In practice, we run only four prompts (Appendix C.4), with the majority of reference queries coming from *repeat-prefill*.

These approaches concentrate reference queries on representations the model naturally produces when processing and reasoning about $\mathcal{C}$. Empirically, in Appendix D, we observe that *self-study* yields the best downstream performance; *context-prefill* and *repeat-prefill* are nearly as good while being faster. We also find that randomly sampling $q_i \sim \mathcal{N}(0, I_d)$ (random-vectors) works, though it lags the other approaches.

“On-policy” queries. Per-layer compaction can induce distribution shift in queries: compacting early layers changes the residual stream seen by later layers, so the queries they produce may differ from those extracted from the unmodified model. To reduce this mismatch, we compact layers sequentially and, for each layer ℓ , extract Q_{ref}^ℓ by running the model with layers $< \ell$ already compacted, then optimize compaction at layer ℓ using these on-policy queries. This yields slight but consistent improvements.

3.2. Constructing β and C_v

Before describing how we construct compact keys C_k , we first explain how we learn β and C_v to satisfy the attention-matching objectives (Eqs. 1–2) given reference queries $Q_{\text{ref}} = [q_1; \dots; q_n] \in \mathbb{R}^{n \times d}$ and compacted C_k . (In one of the variants we consider, we alternate between fitting subsets of C_k and β, C_v .)

Fitting β . Let $\mathbf{m} = [m_1, \dots, m_n]^\top$ be the vector of original attention mass, i.e.,

$$m_i = \sum_{k=1}^T \exp(q_i K_k^\top).$$

Parameterizing $w_j = \exp(\beta_j) > 0$, we would then like

$$m_i \approx \sum_{j=1}^t \exp(q_i (C_k)_j^\top + \beta_j) = \sum_{j=1}^t w_j \exp(q_i (C_k)_j^\top).$$

This results in the following optimization problem

$$\min_{\mathbf{w}: w_j \geq 0} \|\mathbf{A}\mathbf{w} - \mathbf{m}\|_2^2$$

where $\mathbf{A}_{ij} = \exp(q_i (C_k)_j^\top)$ and $\mathbf{w} = [w_1, \dots, w_t]^\top$. We solve for this via nonnegative least squares (NNLS; see Appendix C.2 for implementation details), and then set $\beta_j = \log(w_j)$, clamping w_j to a small positive value if needed. Intuitively, w_j represents how many original keys’ worth of attention mass the compact key $(C_k)_j$ accounts for.

Fitting C_v . Recall that the attention-output matching condition for a query q is given by,

$$\frac{\exp(q K^\top) V}{\sum_{j=1}^T \exp(q K_j^\top)} \approx \left(\frac{\exp(q C_k^\top + \beta)}{\sum_{j=1}^t \exp(q (C_k)_j^\top + \beta_j)} \right) C_v$$

With C_k and β fixed, we can solve for C_v with ordinary least squares. For each reference query q_i we compute

$$y_i = \frac{\exp(q_i K^\top) V}{\sum_{j=1}^T \exp(q_i K_j^\top)} \in \mathbb{R}^{1 \times d},$$

$$x_i = \frac{\exp(q_i C_k^\top + \beta)}{\sum_{j=1}^t \exp(q_i (C_k)_j^\top + \beta_j)} \in \mathbb{R}^{1 \times t}.$$

Stacking into $\mathbf{Y} = [y_1; \dots; y_n] \in \mathbb{R}^{n \times d}$ and $\mathbf{X} = [x_1; \dots; x_n] \in \mathbb{R}^{n \times t}$, we solve for $\mathbf{C}_v$ with

$$\mathbf{C}_v^* = \arg \min_{\mathbf{C}_v} \|\mathbf{X} \mathbf{C}_v - \mathbf{Y}\|_F^2, \quad (3)$$

$$= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}. \quad (4)$$

In summary, given any set of compacted keys $\mathbf{C}_k$, simple linear-algebra routines—least squares and nonnegative least squares—allow us to learn $(\beta, \mathbf{C}_v)$ that minimize ℓ_2 error in the attention mass and attention outputs over the reference queries. Now the main challenge becomes selecting $\mathbf{C}_k$.

3.3. Selecting $\mathbf{C}_k$

We do not have a closed-form solution for constructing $\mathbf{C}_k$ in general. However we found it empirically effective (and efficient) to restrict $\mathbf{C}_k$ to be a subset of the original keys, i.e., $\mathbf{C}_k = \mathbf{K}_{S,:}$ for some index set $S \subset \{1, \dots, T\}$ with $|S| = t$. This allows us to avoid iterative gradient-based optimization. We consider two methods for choosing this subset, which have different efficiency-performance tradeoffs.

Highest attention keys. A simple approach in prior work on KV cache eviction/pruning (Zhang et al., 2023; Li et al., 2024; Kim et al., 2025b) is to retain keys that receive the most attention. In our setting, we measure this under the reference queries. For each reference query $\mathbf{q}_i$, we compute attention weights over the original keys:

$$a_i = \text{softmax}(\mathbf{q}_i \mathbf{K}^\top) \in \mathbb{R}^{1 \times T}.$$

We then aggregate these weights across queries via root mean square over $(a_{1,j}, \dots, a_{n,j})$ to obtain a per-key importance score s_j . We found RMS to be more robust than mean or max aggregation (Appendix F.1).

Orthogonal matching pursuit (OMP) keys. The method above is a fast and effective heuristic in practice. As a more direct alternative, we can explicitly match the attention mass using orthogonal matching pursuit (OMP; Tropp & Gilbert, 2007), which greedily builds $\mathbf{C}_k$ and β to best satisfy Eq. 2.

Define the mass feature matrix $\Phi \in \mathbb{R}^{n \times T}$ with $\Phi_{ij} = \exp(\mathbf{q}_i \mathbf{K}_j^\top)$ and target vector $\mathbf{m}$ with $m_i = \sum_j \Phi_{ij}$. We seek a sparse subset S and weights $\mathbf{w} \geq 0$ minimizing $\|\Phi_{:,S} \mathbf{w} - \mathbf{m}\|_2^2$. OMP selects S greedily: at each step, it adds the key whose column maximally reduces the residual, then refits $\mathbf{w}$ via NNLS (Algorithm 1). This algorithm directly yields $\beta = \log \mathbf{w}$; we then fit $\mathbf{C}_v$ via least squares as above.

While OMP performs best empirically, it is slower than the other methods: greedy selection with NNLS refitting scales at least linearly in t . In practice, selecting multiple keys per step and refitting at intervals reduces compaction time by 4–8 $\times$ with little degradation (Appendix C.1).

Algorithm 1 OMP Key Selection

Require: Original keys $\mathbf{K} \in \mathbb{R}^{T \times d}$, queries $\mathbf{Q} \in \mathbb{R}^{n \times d}$, budget t

Ensure: Indices S , weights $\mathbf{w}$ (with $\beta = \log \mathbf{w}$)

- 1: $\Phi_{ij} \leftarrow \exp(\mathbf{q}_i \mathbf{K}_j^\top / \sqrt{d})$ {Mass feature matrix}
 - 2: $\mathbf{m}_i \leftarrow \sum_{j=1}^T \Phi_{ij}$ {Target mass vector}
 - 3: $\mathbf{r} \leftarrow \mathbf{m}$, $S \leftarrow \emptyset$
 - 4: **for** $k = 1$ **to** t **do**
 - 5: $j^* \leftarrow \arg \max_{j \notin S} (\mathbf{r}^\top \Phi_{:,j})$
 - 6: $S \leftarrow S \cup \{j^*\}$
 - 7: $\mathbf{w} \leftarrow \arg \min_{\mathbf{w} \geq 0} \|\Phi_{:,S} \mathbf{w} - \mathbf{m}\|_2^2$ {NNLS}
 - 8: $\mathbf{r} \leftarrow \mathbf{m} - \Phi_{:,S} \mathbf{w}$
 - 9: **end for**
 - 10: **return** $S, \mathbf{w}$
-

3.4. Nonuniform Compaction

Different attention heads can exhibit different attention patterns (Wu et al., 2025; Xiao et al., 2025; Bick et al., 2025), suggesting that a fixed compaction ratio across all heads is suboptimal. We say that *uniform* compaction uses the same ratio for every KV-head at every layer, whereas *nonuniform* compaction assigns a potentially different ratio to each head and layer.³

To motivate a reusable nonuniform compaction schedule for each model, we first show that attention-head sensitivity is largely input-invariant—although absolute loss varies by example, the relative ranking of head importance remains stable. Figure 2 shows the resulting sensitivity curves for Qwen3-4B, averaged over contexts. These rankings also transfer across datasets; the same qualitative pattern appears on LongHealth (Appendix E).

Precomputed head budgets. This stability enables us to precompute, once per model, a nonuniform compaction schedule—a per-head *share* of the total KV budget—that can be reused across contexts and compaction ratios. Concretely, given the per-head sensitivity curves, we solve a discrete resource allocation problem using a standard greedy exchange algorithm (Algorithm 4). Starting from a uniform allocation across heads, we iteratively swap units of KV budget between heads to minimize loss implied by the sensitivity curves. We repeat this process until no swap yields further improvement. This procedure assumes approximate

³A naïve nonuniform cache implementation would pad all heads to the length of the longest KV-head, increasing the effective context length and partially negating compute savings. However, attention kernels that support variable-length sequences (e.g., FlashAttention (Dao, 2024)) can avoid this overhead by packing per-head KV segments into a flat, variable-length (varlen) representation. With such packing, nonuniform KV caches can achieve the same memory and compute benefits of a uniform cache of the same total size (Feng et al., 2025; Reh, 2024).

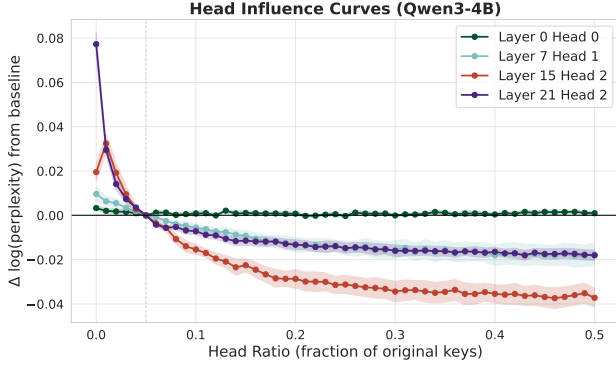


Figure 2. Head sensitivity curves in Qwen3-4B. We fix all KV heads to a baseline compaction ratio of $0.05\times$ and vary the compaction ratio of a single head. We report the change in loss relative to the baseline (lower is better) as a function of the varied head’s compaction ratio. Curves are averaged over 10 QuALITY articles; shaded regions denote ± 1 standard error of the mean across articles. Some heads (e.g., L0H0) are largely insensitive to additional capacity, whereas others (e.g., L15H2) benefit substantially from storing more KV pairs.

separability across heads: we predict the effect of reallocating budget using single-head sensitivity curves measured with other heads held fixed.

The resulting schedule is an efficient model-specific nonuniform allocation procedure that prioritizes capacity for heads most sensitive to compaction. We found this allocation to be robust across instances/datasets, and thus this procedure only needs to be performed once for a given model. A visualization of the learned head budgets is provided in Appendix E.

3.5. Chunked Compaction

To support long contexts, we apply compaction independently to contiguous chunks of the input. Because Attention Matching can select and compact arbitrary token subsets, each chunk can be processed separately and the resulting KV caches concatenated to form a single compacted cache.

We consider two implementations. In *KV-based* chunking, we prefill the full context, slice out the KV states corresponding to each chunk, compact them independently, and merge the compacted chunks. In *text-based* chunking, we instead prefill and compact each chunk in isolation, and then align the resulting keys to their original global positions via a RoPE phase shift before merging. Detailed descriptions of both variants are given in Appendix C.3.

The text-based approach is an approximation because chunks are prefilled without cross-chunk interactions. In practice, we find that KV-based chunking more faithfully preserves model behavior, even when chunks are semantically independent, and therefore use it by default in all experiments, including baselines.

4. Results

We evaluate Attention Matching across a range of compaction strategies, measuring downstream quality and compaction efficiency on long-context benchmarks.

Compared Variants. We evaluate four representative compaction variants that trade off compaction time and downstream quality by varying how reference queries are obtained and how keys are selected. The variants are ordered by performance:

- **AM-OMP:** On-policy Q_{ref} from self-study + repeat-prefill; fit (C_k, β) jointly via OMP; fit C_v by least squares.
- **AM-OMP-fast:** Same as AM-OMP, but with OMP speedups ($k = 4$ keys selected per iteration, $\tau = 2$ iterations between NNLS refits; Appendix C.1).
- **AM-HighestAttnKeys:** On-policy Q_{ref} from self-study + repeat-prefill; select C_k by highest attention; fit β via NNLS; fit C_v by least squares.
- **AM-HighestAttnKeys-fast:** Same as AM-HighestAttnKeys, but with Q_{ref} coming only from repeat-prefill.

With self-study + repeat-prefill, we use at most 50,000 reference queries per KV-head for each context/chunk. Importantly, this does not require a large number of generated tokens: under GQA, each token yields multiple query vectors per KV-head, and the majority of reference queries are obtained via repeat-prefill rather than self-study. On QuALITY, we generate on average approximately 5k self-study tokens and 7k repeat-prefill tokens per context.

Baselines. We compare against (i) **Cartridges** (Eyuboglu et al., 2025), which performs end-to-end gradient-based optimization of a compact latent KV cache for each context; (ii) baselines based on token pruning using attention scores (H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), PyramidKV (Cai et al., 2025), KVzip (Kim et al., 2025b)), instantiated in our one-shot, question-agnostic setting as described in Appendix B.3; and (iii) **summarization**, which replaces the original context with a shorter textual summary, either produced for the full document or by summarizing chunks and concatenating the summaries (Table 5).

Evaluation protocol. We evaluate on QuALITY (Pang et al., 2022), a long-document comprehension benchmark (5–8k tokens; 15–20 questions per context), and LongHealth (Adams et al., 2024), a highly information-dense patient-records QA task (60k tokens per context; 100

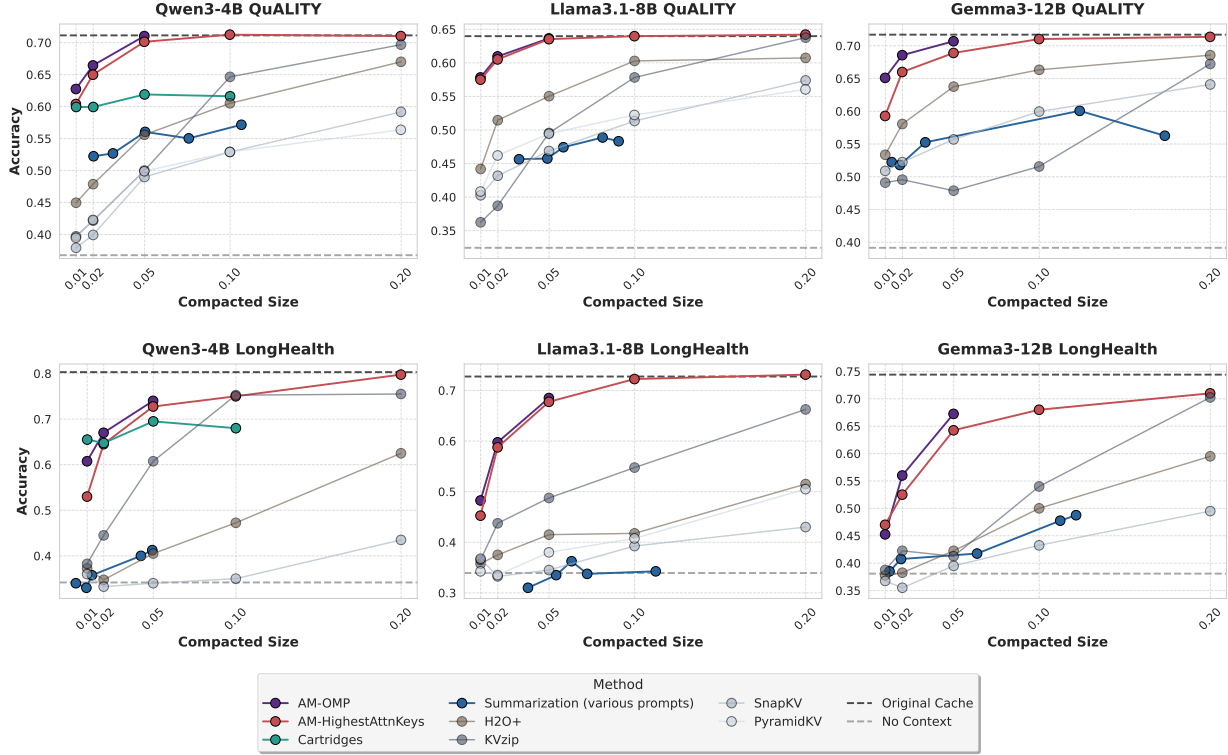


Figure 3. **Accuracy vs. compaction ratio across methods.** We compare AM-OMP and AM-HighestAttentionKeys against Cartridges, summarization, and four prior methods. Evaluations are conducted on QuALITY and LongHealth using Qwen3-4B, Llama3.1-8B, and Gemma3-12B. Attention Matching (AM) consistently outperforms other approaches across compaction ratios, while matching Cartridges’ performance at ultra-high compaction.

questions per context, where each context aggregates 5 patients; 4 contexts total). See Appendix B.2 for a sample context and question.

For each context, we compact once and then answer all associated multiple-choice questions by decoding from the compacted cache. For all methods, we keep the chat template tokens fixed during compaction (e.g., BOS / system-prefix tokens, which can act as attention sinks (Xiao et al., 2024)) and re-attach them after compaction. We report the compacted size t/T with respect to the T article tokens, excluding chat-template tokens. While we found our method is robust to compacting these template tokens, we fix them for a fair comparison with baselines. We apply KV-based chunked compaction on LongHealth with 5 chunks for all methods except for Cartridges.

Our QuALITY evaluation is done over 50 contexts. Since evaluating Cartridges was computationally expensive, we only evaluate it on Qwen, on a fixed 20-context subset selected a priori; scores on this subset showed little deviation from the full set of 50 (Appendix B.2).

4.1. Main Results

We first analyze the trade-off between compaction time and downstream accuracy. Figure 1 plots this for Qwen3-4B on QuALITY at a fixed $50\times$ ratio. (See Figure 10 in the appendix for perplexity results.) Our Attention Matching (AM) methods trace the Pareto frontier and bridge the gap between fast heuristic baselines (e.g., summarization, H2O+, KVzip), which degrade significantly at this ratio, and optimization-heavy methods like Cartridges.

In Figure 3, we sweep the compaction ratio across 3 models on QuALITY and LongHealth. We observe that Attention Matching methods consistently outperform token-eviction baselines and summarization, particularly in the high-compaction regime ($20\times$ – $100\times$). Performance degrades faster on LongHealth, which is highly information-dense (Appendix B.2). Summarization does particularly poorly on this dataset, matching the no-context baseline. KVzip sometimes matches Attention Matching performance, which we attribute to its non-uniform budget outperforming our precomputed budget at certain compaction ratios.

Fast KV Compaction via Attention Matching

Stage	Method	Time (s)
Query generation	context-prefill	7
	repeat-prefill	8
	self-study	139
Key selection	Highest attention	3
	OMP	565
	OMP-fast ($k=4$, refit interval=2)	104
β fitting	NNLS	2.2
Value fitting	Least squares	1.8

Table 1. Wall-clock compaction time breakdown on a 60k-token LONGHEALTH context with Gemma-3-12B on a single H200 GPU.

4.2. Sliding Window Attention

The effectiveness of nonuniform compaction schedules gives some insight into why hybrid architectures with *sliding-window attention* (Child et al., 2019; Beltagy et al., 2020) and only a few global layers can retain much of the performance of full-attention models: even when trained with full attention, many KV-heads naturally specialize to modeling local context, while only a small number of globally attending heads capture the long-range dependencies.

Does this imply that compaction is less effective for these hybrid models? To verify that our improvements still hold on sliding-window models, we evaluate Gemma-3-12B, which has an aggressive 5:1 ratio of sliding-window layers to global layers. We compact only the global-attention layers, leaving the sliding-window layers unchanged. Accordingly, the reported compaction ratios are computed over only the global-attention portion of the KV cache. As shown in Figure 3, we require only slightly more conservative compaction ratios compared to the two full-attention models.

4.3. Compaction Efficiency

We profile the wall-clock compute cost of each component of our method on a 60k-token LONGHEALTH context using Gemma-3-12B (64 full-attention KV heads; head dimension $d_{\text{head}}=256$); see Table 1). All timings are measured on a single H200 GPU. We compute compacted keys and values in FP32 and then cast to BF16 for storage; we do not experiment with quantization in this work.

Query generation dominates runtime in this long-context setting and can likely be substantially optimized. All results here use a single GPU and self-study/repeat-prefill with chunked prefill with chunk size 4096.

Although we only evaluate models up to 12B parameters, we note that KV cache size varies quite little across different size variants of open-source models. For example, for the same sequence length, Qwen3-235B-A22B yields a KV cache only $\sim 30\%$ larger than Qwen3-4B (96,256 vs. 73,728 elements per token). We therefore expect our compaction efficiency results to scale without much additional cost to

Method	Cache Size (%)	Accuracy (%)
Full Context	100.00	71.5
Summarize	4.80	55.2
Summarize $\times$ (0.2 $\times$ AM-OMP)	0.92	55.7
Summarize $\times$ (0.1 $\times$ AM-OMP)	0.46	55.0
Summarize $\times$ (0.05 $\times$ AM-OMP)	0.21	49.2

Table 2. Summarization plus Attention Matching. Applying AM-OMP on top of a summary enables up to $\sim 200\times$ total compaction with accuracy comparable to summarization alone.

much larger mixture of expert models commonly used in deployment.

4.4. Extension: Summarization plus Attention Matching

While summarization rapidly degrades performance on tasks that require extracting knowledge and reasoning across long contexts (Section 4.1), it may be acceptable in cases where not all prior information needs to be retained (for example, when there exist retrieval or search mechanisms), or for filtering out noise or hallucinations.

In Table 2, we demonstrate that Attention Matching works on top of summarization: applying AM-OMP to summarized text achieves $200\times$ compaction (6340 $\rightarrow$ 31 effective tokens on average) with performance comparable to summarization alone, which in our case provides only $\sim 20\times$ compaction.

4.5. Ablations

We’ve introduced several concepts: post-compaction attention biases, fitted values, nonuniform head budgets, and query-sampling methods. In Figure 4, we leave out components of our main AM-OMP method and measure loss on reference generations. We see that every component is necessary to achieve the best results, with the least important being the use of self-study and the use of on-policy queries, and the most important being nonuniform head budgets.

We note that omitting attention biases and retaining the original values for selected keys still yields a reasonable approximation. However, refitting β and C_v via (nonnegative) least squares better aligns with our objective of approximating the original cache, adds little overhead, and consistently improves performance.

5. Related Work

While some prior works approach KV cache reduction as a per-token online problem—continuously evicting or merging tokens at each decoding step to maintain a fixed state size (e.g., Zhang et al., 2023; Oren et al., 2024)—we study compaction as a one-shot operation applied at the moment a context becomes too large. Methods in this setting are able to make globally informed decisions over the full prefix.

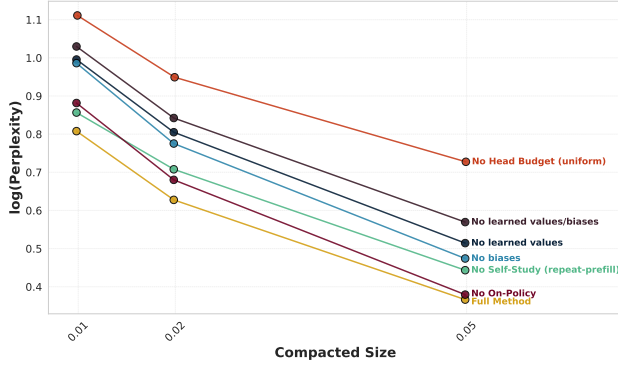


Figure 4. Leave-one-out experiments. We ablate our main AM-OMP method and measure average log-perplexity of generations decoded from the original cache, since this is lower in variance and well-correlated with downstream performance (Appendix F.2). In “no biases,” we first compute OMP as usual and then zero out β , keeping the keys selected by OMP.

Moreover, one-shot compaction can be applied repeatedly to maintain a fixed maximum state size, as in Appendix F.3.

Token-space approaches to handling long context generally involve combinations of retrieval-augmented generation (Lewis et al., 2020), context compaction via summarization or token dropping (Jiang et al., 2023; Li et al., 2023; Anthropic, 2025), and agentic memory management (Packer et al., 2024; Zhang et al., 2025). These approaches are largely orthogonal to latent-space compaction. While summarization rapidly degrades performance on tasks that require extracting knowledge and reasoning across long contexts (Section 4), it may be acceptable in cases where not all prior information needs to be retained (for instance, when there exist retrieval or search mechanisms). The two approaches can be combined with strong performance (Table 2); the most effective memory system may have a mix of retrieval and compaction in both token-space and latent-space.

In addition to the baselines that we evaluate (Zhang et al., 2023; Li et al., 2024; Cai et al., 2025; Kim et al., 2025b), a large body of other work explores token-merging (Wang et al., 2024; Zhang et al., 2024; Liu et al., 2025; Wan et al., 2025) and token-eviction (Oren et al., 2024; Ge et al., 2024; Tang et al., 2024; Chari & Durme, 2025; Łańcucki et al., 2025). The KVPress (Devoto et al., 2025) repository benchmarks over 20 such methods; we include the top-performing approach, KVzip (Kim et al., 2025b), among our baselines.

One line of prior work explores having models *generate* a set of soft tokens or “gist” tokens (Bulatov et al., 2022; Chevalier et al., 2023; Mu et al., 2023) that can be used to replace the original cache. These methods require training or finetuning the model to produce such representations, whereas our approach operates as a post-hoc procedure on any pretrained model without modification.

Our work is also related to Lexico (Kim et al., 2025a), which compresses keys and values by learning a *universal* dictionary per layer and uses orthogonal matching pursuit (OMP) at inference time to store each per-token key/value vector as a sparse code over the learned atoms. Lexico’s objective is ℓ_2 reconstruction of KV vectors whereas we optimize compacted KV vectors for matching attention behavior.

DuoAttention (Xiao et al., 2025) proposes that attention heads be split into a small set of *retrieval* heads that use a full-length KV cache, and *streaming* heads that can run with a constant-length cache containing only attention sinks and recent tokens; they learn this partition via gradient optimization over per-head *gates* that blend full vs. streaming attention on synthetic data, then binarize the gates for deployment. Our nonuniform compaction shares the insight that heads vary in their sensitivity to KV capacity, but we learn continuous per-head budgets via *sensitivity curves* and compact all heads to varying degrees.

The notion of adding a per-token scalar attention bias was suggested in T5 (Raffel et al., 2020) and ALiBi (Press et al., 2022) as a form of positional encoding. Our method also employs attention biases, but for a different purpose: to correct for the attention mass received by the compacted cache.

6. Discussion

Disentangling logical KV length from physical cache size yields a practical primitive operation: KV compaction, which can be invoked on arbitrary portions of context whenever a sequence grows too large. Attention Matching is a natural paradigm for KV compaction because it approximately preserves the two ingredients that determine a block’s contribution under concatenation—attention outputs and attention mass. Moreover, Attention Matching enables efficient algorithms for finding compact keys and values.

Our work still has several limitations. For one, while our approach is orders of magnitude faster than end-to-end gradient-based methods (i.e., Cartridges), our unoptimized OMP + self-study variants still required several minutes for compaction. And while our approach outperforms Cartridges in terms of accuracy at $50\times$ compaction rates, Cartridges outperforms our approach at more extreme ($100\times$) compaction rates on some benchmarks (LongHealth). We attribute this to the fact that gradient-based optimization can search over a wider space of compact representations—in particular, it is not restricted to optimizing intermediate attention outputs or selecting keys from the original cache—which becomes increasingly important as the compaction budget shrinks.

Future work. Beyond speeding up query-generation or OMP-style selection, a promising direction is to move away from subset selection for C_k (e.g., directly optimizing compact keys). Another interesting direction is architectures or training procedures that support compaction as a simple primitive, or explicitly operate over a fixed set of keys and values.

Integrating KV compaction into inference engines (e.g., RadixAttention-style prefix caching, varlen KV packing, and disaggregated compaction) and combining token-space retrieval/summarization with latent-space compaction are important systems directions that still require significant effort.

Finally, while our experiments focus on one-shot compaction of a fixed context, the problem of maintaining memory over long-horizon interactions is another very natural setting for KV compaction. A promising application is *online compaction*, where we compact the KV cache mid-trajectory to support arbitrarily long-horizon generation under a fixed physical memory budget. Appendix F.3 provides a preliminary result showing that repeated context compaction via Attention Matching preserves reasoning performance on AIME. This is especially promising for long-horizon agentic settings such as coding agents, where tool call outputs and execution traces can take up a significant amount of context. Online latent compaction could be a principled alternative or complement to turn-dropping and summarization.

7. Conclusion

We study Attention Matching as an objective for fast latent-space compaction. Our methods significantly improve the Pareto frontier of compaction cost versus quality.

Acknowledgements

We would like to thank Ani Nrusimha, Alicia Li, Sabri Eyuboglu, Zifan Wang, Jyo Pari, Oliver Sieberling, and Tarushii Goel for their valuable discussions and feedback. This work was supported by the National Science Foundation under CAREER Award No. 2441872 and MIT-IBM Watson AI Lab. This research was partly supported by the Prof. JCR Licklider Endowed UROP Fund for MIT UROP research.

References

Adams, L., Busch, F., Han, T., Excoffier, J.-B., Ortala, M., Löser, A., Aerts, H. J., Kather, J. N., Truhn, D., and Bressen, K. LongHealth: A question answering benchmark with long clinical documents, 2024. URL <https://arxiv.org/abs/2401.14490>.

Anthropic. Effective context engineering for AI agents, 2025. URL <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.

Bick, A., Xing, E. P., and Gu, A. Understanding the skill gap in recurrent models: The role of the Gather-and-Aggregate mechanism. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=hWYisUBbp7>.

Braverman, M., Chen, X., Kakade, S., Narasimhan, K., Zhang, C., and Zhang, Y. Calibration, entropy rates, and memory in language models. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

Bulatov, A., Kuratov, Y., and Burtsev, M. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/47e288629a6996a17ce50b90a056a0e1-Paper-Conference.pdf.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., and Xiao, W. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=ayi7qezU87>.

Cao, S., Valiant, G., and Liang, P. On the entropy calibration of language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=CGLoEvC11I>.

Chari, V. and Durme, B. V. Compactor: Calibrated query-agnostic KV cache compression with approximate leverage scores, 2025. URL <https://arxiv.org/abs/2507.08143>.

Chevalier, A., Wettig, A., Ajith, A., and Chen, D. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.232/>.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers, 2019. URL <https://arxiv.org/abs/1904.10509>.

Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL

- <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.
- Devoto, A., Jeblick, M., and Jégou, S. Expected Attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025. URL <https://arxiv.org/abs/2510.00636>.
- Dong, J., Feng, B., Guessous, D., Liang, Y., and He, H. FlexAttention: A programming model for generating fused attention variants. In *Eighth Conference on Machine Learning and Systems*, 2025. URL <https://openreview.net/forum?id=2QMYV4bA0R>.
- Eyuboglu, S., Ehrlich, R., Arora, S., Guha, N., Zinsley, D., Liu, E., Tennien, W., Rudra, A., Zou, J., Mirhoseini, A., and Re, C. Cartridges: Lightweight and general-purpose long context representations via self-study, 2025. URL <https://arxiv.org/abs/2506.06266>.
- Feng, Y., Lv, J., Cao, Y., Xie, X., and Zhou, S. K. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=tcisuhGsQZ>.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=uNrFpDPMYo>.
- Gemma Team. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., and Qiu, L. LLM-Lingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.825/>.
- Juravsky, J., Brown, B., Ehrlich, R. S., Fu, D. Y., Re, C., and Mirhoseini, A. Hydragen: High-throughput LLM inference with shared prefixes. In *Workshop on Efficient Systems for Foundation Models II @ ICML2024*, 2024. URL <https://openreview.net/forum?id=Ye3ia34Fpp>.
- Kim, J., Park, J., Cho, J., and Papailiopoulous, D. Lexico: Extreme KV cache compression via sparse coding over universal dictionaries. In *Forty-second International Conference on Machine Learning*, 2025a. URL <https://openreview.net/forum?id=Yh9vxlxnjA>.
- Kim, J.-H., Kim, J., Kwon, S., Lee, J. W., Yun, S., and Song, H. O. KVzip: Query-agnostic KV cache compression with context reconstruction. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025b. URL <https://openreview.net/forum?id=JFygzwx8SJ>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Łańcucki, A., Staniszewski, K., Nawrot, P., and Ponti, E. Inference-time hyper-scaling with KV cache compression. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=8ZiElzQxf1>.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
- Li, X. L. and Liang, P. Prefix-Tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 2021. URL <https://aclanthology.org/2021.acl-long.353/>.
- Li, Y., Dong, B., Guerin, F., and Lin, C. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://aclanthology.org/2023.emnlp-main.391/>.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/28ab418242603

- e0f7323e54185d19bde-Paper-Conference.pdf.
- Liu, X., Wang, X., Liu, P., and Tang, G. ZSMerge: Zero-shot KV cache compression for memory-efficient long-context LLMs, 2025. URL <https://arxiv.org/abs/2503.10714>.
- Llama Team. The Llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Mu, J., Li, X., and Goodman, N. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/3d77c6dcc7f143aa2154e7f4d5e22d68-Paper-Conference.pdf.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candes, E., and Hashimoto, T. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. URL <https://aclanthology.org/2025.emnlp-main.1025/>.
- OpenAI. Context engineering - short-term memory management with sessions from OpenAI agents SDK, 2025. URL https://cookbook.openai.com/examples/agents_sdk/session_memory. OpenAI Cookbook. Page author: Emre Okcular.
- Oren, M., Hassid, M., Yarden, N., Adi, Y., and Schwartz, R. Transformers are multi-state RNNs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. URL <https://aclanthology.org/2024.emnlp-main.1043/>.
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. MemGPT: Towards LLMs as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.
- Pang, R. Y., Parrish, A., Joshi, N., Nangia, N., Phang, J., Chen, A., Padmakumar, V., Ma, J., Thompson, J., He, H., and Bowman, S. QuALITY: Question answering with long input texts, yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2022. URL <https://aclanthology.org/2022.naacl-main.391/>.
- Press, O., Smith, N., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=R8sQPpGCv0>.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- Rehg, I. KV-Compress: Paged KV-cache compression with variable compression rates per attention head, 2024. URL <https://arxiv.org/abs/2410.00161>.
- Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., and Han, S. QUEST: query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Tropp, J. A. and Gilbert, A. C. Signal recovery from random measurements via orthogonal matching pursuit. *IEEE Transactions on information theory*, 53(12):4655–4666, 2007.
- Wan, Z., Wu, X., Zhang, Y., Xin, Y., Tao, C., Zhu, Z., Wang, X., Luo, S., Xiong, J., Wang, L., and Zhang, M. D2O: Dynamic discriminative operations for efficient long-context inference of large language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=HzBfoUdjHt>.
- Wang, Z., Jin, B., Yu, Z., and Zhang, M. Model tells you where to merge: Adaptive KV cache merging for LLMs on long-context tasks, 2024. URL <https://arxiv.org/abs/2407.08454>.
- Wu, W., Wang, Y., Xiao, G., Peng, H., and Fu, Y. Retrieval head mechanistically explains long-context factuality. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=EytBpUGB1Z>.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.
- Xiao, G., Tang, J., Zuo, J., junxian guo, Yang, S., Tang, H., Fu, Y., and Han, S. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=cFu7ze7xUm>.

Ye, Z., Lai, R., Lu, B.-R., Lin, C.-Y., Zheng, S., Chen, L., Chen, T., and Ceze, L. Cascade Inference: Memory bandwidth efficient shared prefix batch decoding, 2024. URL <https://flashinfer.ai/2024/02/02/cascade-inference.html>.

Zhang, A. L., Kraska, T., and Khattab, O. Recursive language models, 2025. URL <https://arxiv.org/abs/2512.24601>.

Zhang, Y., Du, Y., Luo, G., Zhong, Y., Zhang, Z., Liu, S., and Ji, R. CaM: Cache merging for memory-efficient LLMs inference. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://proceedings.mlr.press/v235/zhang24n.html>.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z. A., and Chen, B. H2O: Heavy-Hitter Oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/6ceefa7b15572587b78ecfceb2827f8-Paper-Conference.pdf.

A. Attention Matching Details

This appendix restates the attention-matching objectives from Section 2 in more detail.

A.1. Notation

Fix a single layer and KV-head with head dimension d . Let the original keys and values be

$$\mathbf{K} \in \mathbb{R}^{T \times d}, \quad \mathbf{V} \in \mathbb{R}^{T \times d},$$

and let the compacted keys, values, and per-token bias be

$$\mathbf{C}_k \in \mathbb{R}^{t \times d}, \quad \mathbf{C}_v \in \mathbb{R}^{t \times d}, \quad \boldsymbol{\beta} \in \mathbb{R}^t.$$

Let $\mathbf{Q}_{\text{ref}} \in \mathbb{R}^{n \times d}$ denote the reference query matrix whose i th row is $\mathbf{q}_i$.

Attention operators. For any key/value block $(\mathbf{K}, \mathbf{V})$ and query $\mathbf{q}$, define the scaled logits

$$\ell(\mathbf{q}; \mathbf{K}) = \frac{1}{\sqrt{d}} \mathbf{q} \mathbf{K}^\top$$

and the unnormalized mass

$$\text{Mass}(\mathbf{q}; \mathbf{K}) = \sum_j \exp(\ell(\mathbf{q}; \mathbf{K})_j).$$

Define the locally normalized attention output

$$\text{Attn}(\mathbf{q}; \mathbf{K}, \mathbf{V}) = \frac{\exp(\ell(\mathbf{q}; \mathbf{K})) \mathbf{V}}{\text{Mass}(\mathbf{q}; \mathbf{K})}.$$

For compacted parameters $(\mathbf{C}_k, \boldsymbol{\beta}, \mathbf{C}_v)$ we use

$$\ell(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta}) = \frac{1}{\sqrt{d}} \mathbf{q} \mathbf{C}_k^\top + \boldsymbol{\beta}, \quad \text{Mass}(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta}) = \sum_j \exp(\ell(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta})_j),$$

and

$$\text{Attn}(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta}, \mathbf{C}_v) = \frac{\exp(\ell(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta})) \mathbf{C}_v}{\text{Mass}(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta})}.$$

A.2. Mass-preserving Attention Matching

We justify replacing attention over $[\mathbf{K}; \mathbf{K}_{\text{fixed}}]$ with the two objectives (1)–(2). However, we observe that attention over 2 subsets of indices decomposes into a mixture whose weights are determined by *unnormalized attention mass*.

Let $(\mathbf{K}, \mathbf{V})$ be the block we compact and $(\mathbf{K}_{\text{fixed}}, \mathbf{V}_{\text{fixed}})$ be any other block (e.g., chat-template tokens and/or future tokens). For any query $\mathbf{q}$,

$$\begin{aligned} \text{Attn}\left(\mathbf{q}; \begin{bmatrix} \mathbf{K} \\ \mathbf{K}_{\text{fixed}} \end{bmatrix}, \begin{bmatrix} \mathbf{V} \\ \mathbf{V}_{\text{fixed}} \end{bmatrix}\right) &= \frac{\text{Mass}(\mathbf{q}; \mathbf{K})}{\text{Mass}(\mathbf{q}; \mathbf{K}) + \text{Mass}(\mathbf{q}; \mathbf{K}_{\text{fixed}})} \text{Attn}(\mathbf{q}; \mathbf{K}, \mathbf{V}) \\ &\quad + \frac{\text{Mass}(\mathbf{q}; \mathbf{K}_{\text{fixed}})}{\text{Mass}(\mathbf{q}; \mathbf{K}) + \text{Mass}(\mathbf{q}; \mathbf{K}_{\text{fixed}})} \text{Attn}(\mathbf{q}; \mathbf{K}_{\text{fixed}}, \mathbf{V}_{\text{fixed}}). \end{aligned} \tag{5}$$

Now suppose we replace $(\mathbf{K}, \mathbf{V})$ by compact parameters $(\mathbf{C}_k, \boldsymbol{\beta}, \mathbf{C}_v)$. If for queries of interest we ensure

$$\text{Attn}(\mathbf{q}; \mathbf{K}, \mathbf{V}) \approx \text{Attn}(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta}, \mathbf{C}_v) \quad \text{and} \quad \text{Mass}(\mathbf{q}; \mathbf{K}) \approx \text{Mass}(\mathbf{q}; \mathbf{C}_k, \boldsymbol{\beta}),$$

then the mixture identity (5) implies that the full attention output is preserved even when *arbitrary* $\mathbf{K}_{\text{fixed}}, \mathbf{V}_{\text{fixed}}$ are appended later, because both mixture weights and the compacted-block contribution match. This means we can use objectives (1)–(2) to do one-shot compaction without knowing future keys/values.

Why biases matter. If C_k is chosen as a subset of K and we do not use attention biases, then $\text{Mass}(\mathbf{q}; C_k) \leq \text{Mass}(\mathbf{q}; K)$ for every $\mathbf{q}$, so the compacted block systematically receives too little global weight in (5). A bias β introduces multiplicative weights $\exp(\beta_j)$ so that each retained key can represent the mass of many removed keys, making mass matching feasible. Furthermore, even if C_k is not chosen as a subset of K , there are cases where the bias is necessary to match attention mass, such as $\mathbf{q} = \mathbf{0}$, which requires matching T vs t . For these reasons, introducing attention biases after compaction is very natural.

Stable computation. In implementation, we evaluate Mass and Attn using the standard per-query max-shift for numerical stability. For logits ℓ with $s = \max_j \ell_j$, we compute $\sum_j \exp(\ell_j) = \exp(s) \sum_j \exp(\ell_j - s)$ and similarly for the numerator of Attn. This does not change any of the identities above. For OMP key selection, we operate on shifted features $\exp(\ell - s)$ and shifted targets $\sum_j \exp(\ell - s)$.

B. Experimental Details

B.1. Models

We evaluate Qwen3-4B (Qwen Team, 2025), Llama-3.1-8B-Instruct (Llama Team, 2024), and Gemma-3-12b-it (Gemma Team, 2025). We use a maximum generation length of 2048 tokens and each model’s default decoding settings (temperature, top- k , and top- p). For long-context benchmarks (LongHealth), we use the Qwen3-4B-Instruct-2507 variant instead of Qwen3-4B because it has a longer native sequence length.

B.2. Datasets

Our default evaluation dataset is **QuALITY** (Pang et al., 2022), a question-answering dataset testing comprehension of 5-7k-token passages. For our evaluations, we first compact the article context and then evaluate performance on the resulting QA task by batched decoding over the associated questions. We believe that pairing each article with many comprehension-focused multiple-choice questions makes QuALITY a controlled and informative benchmark for compaction.

We evaluate performance over the first 50 articles in the validation set (894 questions). Due to the computational cost of Cartridges (~5 hours per context), we only evaluate it on Qwen3-4B, on the first 20 articles (360 questions). Original-cache accuracy on this subset (75.0%) slightly exceeds that of the full set (72.1%).

We evaluate chunked compaction on **LongHealth** (Adams et al., 2024) by concatenating 5 patient records per context. This yields 4 contexts of roughly 60k tokens each, with 100 multiple-choice questions per context. We compact each context in 5 chunks before evaluating QA performance. While Eyuboglu et al. (2025) concatenate 10 patient records, we found that our evaluated base models’ performance degraded beyond ~100k tokens, which we attribute to limitations in long-context extrapolation of the model rather than the compaction procedure. In contrast, Cartridges optimizes a single latent representation across chunks and is thus not sensitive to long-context brittleness in the base model.

We show an example context snippet and question below for QuALITY and LongHealth. These tasks can involve heavy information-extraction and reasoning across the long context, making them an ideal and difficult testbed for compaction.

QuALITY

Context (excerpt):

THE GIRL IN HIS MIND By ROBERT F. YOUNG

Transcriber’s Note: This etext was produced from Worlds of Tomorrow April 1963

Extensive research did not uncover any evidence that the U.S. copyright on this publication was renewed.

Every man’s mind is a universe with countless places in which he can hide—even from himself!

... (5-7k token article)

Sample Question:

Why does Deirdre get so upset when Blake Past suggests she go to prom with the young man?

- A) Because Blake is trying to guilt Deirdre into going with the young man by telling her that it’ll ease her conscience.
- B) Because Deirdre has fallen in love with Blake, despite his age, and wants him to take her to the prom.
- C) Because Blake is acting like he’s her father, which is a sensitive topic for Deirdre because she lost her real parents.
- D) Because the young man gave up his right arm in order to afford tickets to the prom, and this disgusts Deirdre.

LongHealth**Context (excerpt):**

Dear colleague,

We wish to provide an update regarding Mrs. Anna Sample, born on 01.01.1970. She was admitted to our clinic from 01/01/2017 to 01/02/2017.

Diagnosis: Diffuse large B-cell lymphoma of germinal center type; ID 01/2017

- Ann-Arbor: Stage IV
 - R-IPI: 2 (LDH, stage)
 - CNS-IPI: 2
 - Histology: Aggressive B-NHL (DLBCL, NOS); no evidence of t(14;18) translocation. Ki-67 at 40%. Positive reaction to MUM1, numerous CD68-positive macrophages. Negative reaction to ALK1 and TdT.
 - cMRI: Chronic inflammatory lesions suggestive of Multiple Sclerosis (MS)
 - CSF: no evidence of malignancy
 - Bone marrow aspiration: no infiltration from the pre-existing lymphoma.
- Current treatment:
- Initiated R-Pola-CHP regimen q21
- Polatuzumab vedotin: 1.8mg/kg on Day 1.
- ... (10-12k tokens per patient; 5 patients)

Sample Question:

Mrs. Sample received multiple radiologic examinations. In which order did she receive them?

- A) MR spine > MR Head > CT Thoracic Spine > CT Whole Body
- B) MR Brain > MR thoracic/lumbar spine > CT chest/abdomen/pelvis > CT Thoracic Spine > CT Thoracic Spine > CT chest/abdomen/pelvis > Abdominal ultrasound > MR Spine > Whole-body PET/CT > Liver MRI
- C) MR cervical spine > MR Head > CT Whole Body > CT Thoracic Spine
- D) MR Brain > CT Thoracic Spine > MR cervical spine > CT Whole Body
- E) CT Thoracic Spine > CT Whole Body > MR cervical spine > MR Head

B.3. Baselines

Cartridges. We run experiments using the Cartridges repository (Eyuboglu et al., 2025). For each QuALITY article, we synthesize 32,768 self-study samples (25–30M tokens) using the default settings and train for 1 epoch with the default learning rate (2×10^{-2}). We replicate the same order of magnitude of training compute for a fair comparison (Table 3). We use the same evaluation setup as in our other experiments. Optimizing training or using more diverse self-study samples may greatly speed up results; we use the default settings provided in the open-source repository.

Dataset	Context	Self-Study Tokens	H100-hours
LongHealth-5	60k	100–150M	~15
Quality	7k	25–30M	~5
LongHealth-10 (Original)	120k	300M	-

Table 3. **Training compute for Cartridges.** We sample approximately the same number of self-study tokens per context token as in the original Cartridges implementation.

H2O, SnapKV, PyramidKV, and KVzip. We evaluate H2O, SnapKV, PyramidKV, and KVzip (Zhang et al., 2023; Li et al., 2024; Cai et al., 2025; Kim et al., 2025b). These methods select a subset of keys based on highest attention scores under a chosen set of queries.

H2O was originally proposed as an online eviction policy; here we use its one-shot compaction analogue, which we term **H2O+**. Concretely, during context prefill we collect all queries for each KV-head position and then select the top- k tokens per head by attention score, rather than evicting tokens sequentially based on only partial query sets. Moreover, because we compact only the content portion of the context—keeping chat-template tokens and the question portion fixed—we do not explicitly retain all recent tokens as in the original paper. Finally, we use the GQA variant of this method (and of the

Method	Queries	Key selection	β	Values	Nonuniform
H2O+	context-prefill	HighestAttnKeys	none	direct	none
SnapKV (agnostic)	mock-question	HighestAttnKeys	none	direct	none
PyramidKV (agnostic)	mock-question	HighestAttnKeys	none	direct	linearly decreasing
KVzip-uniform	repeat-prefill	HighestAttnKeys	none	direct	none
KVzip	repeat-prefill	GlobalHighestAttnKeys	none	direct	global top- k

Table 4. **Baseline configurations.** “Mock-question” denotes a single synthetic question generated via self-study to instantiate originally question-aware methods in a question-agnostic setting. “Direct” values indicate that retained keys keep their original values (i.e., no value fitting with least squares).

methods below): instead of duplicating KV heads before compaction (as some implementations do), we aggregate queries from all query heads that attend to a given KV head into a shared query set for that KV head.

SnapKV and PyramidKV are question-aware in their original form (they compact after observing the downstream question). To make them question-agnostic and comparable to our setting, we instead generate a single mock question via self-study and use it to construct the query set used for key scoring. We use max-pooling with a kernel-size of 7, consistent with original hyperparameters.

Table 4 summarizes the design choices for each baseline.

DuoAttention. For Llama-3.1-8B-Instruct, we additionally attempted to compare against DuoAttention (Xiao et al., 2025), which supports this model. We implement DuoAttention by partitioning heads into *streaming* and *retrieval* heads: for streaming heads we evict the article portion of the KV cache, while for retrieval heads we retain the full KV cache. The overall compaction ratio determines the fraction of heads assigned to each category. However, at the compaction ratios we evaluated (retaining up to $0.2\times$ the original cache), DuoAttention performed poorly, so we omit it from the main figure.

Summarization. As a baseline, we summarize the article text and use the summary in place of the original article for downstream evaluation. We evaluate several prompt variants shown in Table 5.

Name	Prompt
summarize	Summarize the following text: {article_text}
summarize_indepth	Summary: Summarize the following text in depth: {article_text}
summarize_keypoints_questions	Summary: Summarize the following text, providing all key points that might be necessary to answer comprehension questions: {article_text}
summarize_concise	Summary: Summarize the following text concisely: {article_text}
summarize_very_concise	Summary: Summarize the following text very concisely: {article_text}
	Summary:

Table 5. **Summarization prompts.**

C. Algorithmic Implementation Details

We implement each of our per-head algorithm sequentially, iterating through each KV-head in the model. We did not find major speedups through batching computation across heads.

C.1. OMP Speedups

The periodic-refit OMP key selection algorithm is shown in Algorithm 2. It introduces two hyperparameters, k and τ : k is the number of keys selected per greedy iteration, and τ is the number of iterations between NNLS refits (i.e., we recompute weights once every $k\tau$ newly added keys). Our main algorithm implicitly uses $k = 1, \tau = 1$, and we use $k = 4$ and $\tau = 2$ in our fast variant.

Algorithm 2 OMP Keys Selection (Periodic Refit)

Require: Original keys $\mathbf{K} \in \mathbb{R}^{T \times d}$, queries $\mathbf{Q} \in \mathbb{R}^{n \times d}$, budget t , top- k k , NNLS interval τ

Ensure: Indices S , weights $\mathbf{w}$ (where $\beta = \log \mathbf{w}$)

```

1:  $\Phi_{ij} \leftarrow \exp\left(\frac{1}{\sqrt{d}} \mathbf{q}_i \mathbf{K}_j^\top\right)$  {Mass feature matrix}
2:  $\mathbf{m}_i \leftarrow \sum_{j=1}^T \Phi_{ij}$  {Target mass vector}
3:  $S \leftarrow \emptyset, \mathbf{w} \leftarrow [], \mathbf{r} \leftarrow \mathbf{m}$ 
4: for  $u = 1$  to  $t$  do
5:    $c_j \leftarrow \mathbf{r}^\top \Phi_{:j} \quad \forall j \notin S$  {Correlation scores}
6:    $J^* \leftarrow \text{TopK}(\{c_j\}_{j \notin S}, k)$  {Select top- $k$  new keys}
7:    $S \leftarrow S \cup J^*$ 
8:   if  $u \bmod \tau = 0$  or  $u = t$  then
9:      $\mathbf{w} \leftarrow \arg \min_{\mathbf{w} \geq 0} \|\Phi_{:S} \mathbf{w} - \mathbf{m}\|_2^2$  {Solve NNLS}
10:     $\mathbf{r} \leftarrow \mathbf{m} - \Phi_{:S} \mathbf{w}$  {Update residual}
11:   end if
12: end for
13: return  $S, \mathbf{w}$ 

```

C.2. Linear Algebra Subroutines

Numerics. We compute C_k, β , and C_v in FP32, then cast to BF16 for storage and subsequent use.

Least Squares. We evaluated three solvers for the least-squares problems in Eq. 3: `torch.linalg.lstsq`, `torch.linalg.pinv`, and a Cholesky-based approach using `torch.linalg.cholesky` with `torch.cholesky_solve`. Concretely, we compute

1. `torch.linalg.lstsq`: We directly compute the solution to the optimization problem of $\arg \min_M \|XM - Y\|_F^2$, as $M = \text{torch.linalg.lstsq}(X, Y)$. On CUDA, PyTorch uses the `gels` driver, performing QR decomposition on $X = QR$, transforming the target Y by left multiplication with $C = Q^T$, then solves the triangular problem of $RM = C$.
2. `torch.linalg.pinv`: We compute the Moore-Penrose pseudo-inverse explicitly and then right multiply by Y , i.e. $M = \text{torch.linalg.pinv}(X)Y$.
3. `torch.linalg.cholesky`: We solve the normal equations $X^T XM = X^T Y$ by computing the Cholesky decomposition of the symmetric positive-definite matrix $A = X^T X$:

$$LL^T = X^T X$$

where $L = \text{torch.linalg.cholesky}(A)$. The solution is obtained by solving the two-step triangular system $LZ = X^T Y$ and $L^T M = Z$ via `torch.cholesky_solve`.

Across our experiments, solution quality ranked `lstsq > cholesky > pinv`, though the difference was minimal. In terms of runtime, `cholesky` was fastest, followed by `lstsq`, then `pinv`. We therefore use `torch.linalg.lstsq`

in all experiments. We also tested ℓ_2 regularization when estimating β and C_v by computing the regularized solution $(X^T X + \lambda I)^{-1}$, but found that it degraded performance across all positive values of λ .

Nonnegative Least Squares (NNLS). We implement an NNLS solver with projected gradient descent. When `iters=0`, we compute an unconstrained least-squares solution with `torch.linalg.lstsq` and then clamp to enforce $B \geq \epsilon$ (and optionally $B \leq u$). For `iters>0`, we warm-start from this clamped solution and run projected gradient descent for `iters` steps on $\frac{1}{2}\|MB - y\|_2^2$, using a fixed step size $1/L$ where $L \approx \|M\|_2^2$ is estimated via a few power-iteration steps.

Algorithm 3 describes the NNLS solver used throughout. We initialize from an unconstrained least-squares solution followed by clamping, and (when `iters>0`) refine the solution using projected gradient descent on $\frac{1}{2}\|MB - y\|_2^2$ with a fixed step size.

Algorithm 3 NNLS via Projected Gradient Descent

Require: Matrix $M \in \mathbb{R}^{n \times p}$, target $y \in \mathbb{R}^n$, iterations `iters`, lower bound $\epsilon \geq 0$, optional upper bound u

Ensure: Nonnegative weights $B \in \mathbb{R}^p$

```

1:  $B^{(0)} \leftarrow \arg \min_B \|MB - y\|_2^2$  {Unconstrained least squares}
2:  $B^{(0)} \leftarrow \max(B^{(0)}, \epsilon)$  {Clamp to enforce  $B \geq \epsilon$ }
3: if upper bound  $u$  is specified then
4:    $B^{(0)} \leftarrow \min(B^{(0)}, u)$ 
5: end if
6: if iters = 0 then
7:   return  $B^{(0)}$ 
8: end if
9: Estimate  $L \approx \|M\|_2^2$  via power iteration
10:  $\eta \leftarrow 1/L$  {Fixed step size}
11: for  $t = 0$  to iters do
12:    $g^{(t)} \leftarrow M^T (MB^{(t)} - y)$  {Gradient}
13:    $\tilde{B} \leftarrow B^{(t)} - \eta g^{(t)}$  {Gradient step}
14:    $B^{(t+1)} \leftarrow \max(\tilde{B}, \epsilon)$  {Projection}
15:   if upper bound  $u$  is specified then
16:      $B^{(t+1)} \leftarrow \min(B^{(t+1)}, u)$ 
17:   end if
18: end for
19: return  $B^{(\text{iters})}$ 
    
```

For OMP Keys, we set `iters` = 0 as we observed that the unconstrained solution rarely yields negative weights. For Highest Attention Keys, we use `iters`=2.

Stabilizing β . Our pipeline first fits β to match attention mass and then fits C_v to match attention outputs. A potential failure mode of this two-stage procedure is that mass matching can assign extremely small weights to some selected keys (i.e., very negative β , or effectively $\beta = -\infty$). Such keys may have little effect on the mass objective yet still be useful for reducing attention-output error; however, once β is very negative, the corresponding key cannot contribute to the attention output, regardless of C_v .

To mitigate this, we apply simple stability constraints on β . For *Highest Attention Keys*, we replace NNLS with a bounded least-squares over $w = \exp(\beta)$, enforcing $e^{-3} \leq w_j \leq e^3$ (equivalently, $\beta_j \in [-3, 3]$). The projected gradient method above supports these box constraints directly.

For *OMP Keys*, which rarely produces extreme β values, we instead use a lightweight pruning rule: after greedy selection, we discard any selected keys with $\beta < -7$ and continue selection until all retained keys satisfy $\beta \geq -7$. We also cap large biases during weight fitting by enforcing $w_j \leq e^7$ (equivalently, $\beta_j \leq 7$). In practice, these OMP safeguards did not yield noticeable performance gains, since OMP rarely assigns very low or high β values; we nonetheless include them as simple, theoretically sound stabilizers.

C.3. Chunked Compaction Details

Here we provide implementation details for chunked compaction. Given a document split into N contiguous chunks, we apply per-layer/per-head compaction independently to each chunk and concatenate the resulting compacted KV segments to form the final cache:

$$[\text{prefix}] + [\text{compacted chunk}_1] + \dots + [\text{compacted chunk}_N] + [\text{suffix}],$$

where the prefix and suffix are kept fixed with $\beta = 0$, and each chunk contributes learned (C_k, β, C_v) .

We consider two variants: KV-based and text-based chunking.

KV-based chunking prefills the full context once, slices out each chunk’s KV states, compacts those tensors, and concatenates the compacted chunks. To generate self-study reference queries for chunk i , we construct a cache by concatenating the corresponding KV tensors:

$$[\text{prefix}] + [\text{chunk}_i] + [\text{suffix}],$$

leaving any sliding-window layers unchanged (copied from the full prefill). When prefilling self-study conversations to extract reference queries, we continue positional indexing from the original sequence length so that appended tokens receive the same RoPE phases as in the uncompacted run. These choices preserve the global positional semantics of the original sequence: although the physical cache size is smaller, its logical length and RoPE phases match those of the uncompacted run.

Text-based chunking prefills and compacts each chunk in isolation as $[\text{local prefix}] + [\text{local chunk}_i] + [\text{local suffix}]$, using chunk-local positions starting at 0, and then applies a uniform RoPE phase shift to the compacted keys to align them to the chunk’s original global offset in the full context (i.e., a rotation by $\Delta = p_{\text{global}} - p_{\text{local}}$). For models with sliding-window attention (e.g., Gemma), we handle sliding layers by instantiating the final cache’s sliding-window KV from the last chunk (including the global suffix) with the same positional shift applied.

C.4. Self-study

In our implementation of self-study, we run two “instances” of the model:

1. **Conversation-starter generation.** Instance A is prompted with $\mathcal{C}$ and a seed prompt (e.g., “Write 3 different questions separated by newlines that test understanding of the context.”). We sample

$$a \sim \text{LM}(\cdot \mid \mathcal{C}, \text{seed}),$$

then post-process a into m separate conversation starters $(a_i)_{i=1}^m$.

2. **Response generation.** For each a_i , we sample from instance B a response

$$b_i \sim \text{LM}(\cdot \mid \mathcal{C}, a_i).$$

During instance B’s prefill on a_i and decoding of b_i , we record, for each attention head, all query vectors used by the model.

We include a set of fixed prompts (e.g., “Aggregate all key facts mentioned in the context.”) by directly setting a to the prompt string and sampling the corresponding response from instance B.

We use vLLM (Kwon et al., 2023) to generate self-study conversations, and then we prefill these conversations with a forward hook that extracts query vectors at each KV-head. To control the number of extracted queries, we apply reservoir sampling and cap the buffer at 50,000 query vectors per KV-head. In practice, our self-study configuration typically yields fewer than this cap (avg. $\sim 16,000$ per head for QuALITY).

For on-policy self-study and repeat-prefill, we first generate each self-study sequence and prefill the first layer to extract its queries. Then, for each layer L , we prefill layer L using the compacted cache for layers 0 through $L - 1$, and extract the query vectors at layer L .

Name	Prompt
3-question	Write 3 questions that test understanding of different parts of the context. Answer with just the 3 questions and options (do not say the correct answer), each one separated with 2 newlines.
summarize	Summarize the main points of the context.
structure_json	Structure the information in JSON form and include all important details like dates, times, names, and numerical values.
aggregate	Aggregate all the key facts mentioned in the context.

Table 6. Prompt templates used in self-study.

Prompts. We use the prompt templates in Table 6.

We obtain one conversation starter directly from each of summarize, structure_json, and aggregate. For 3-question, we sample a response from instance A and split the generated output into three conversations starters. We then sample one response per starter.

D. Impact of Reference Queries

We present uniform compaction results under different reference query generation strategies in Figure 5. We evaluate $\log(\text{Perplexity})$ of the original cache’s answers on 20 QuALITY instances (357 questions). We plot this rather than downstream accuracy because it has lower variance and correlates well with downstream accuracy.

For random vectors, we sample IID from a normal distribution with mean 0 and standard deviation 1, and then apply q -norm scaling for the respective attention heads. We omit this step in random-vectors-no-qnorm. In ss-5k and ss-plus-repeat-5k, we subsample 5,000 query vectors uniformly at random after running the corresponding query generation method.

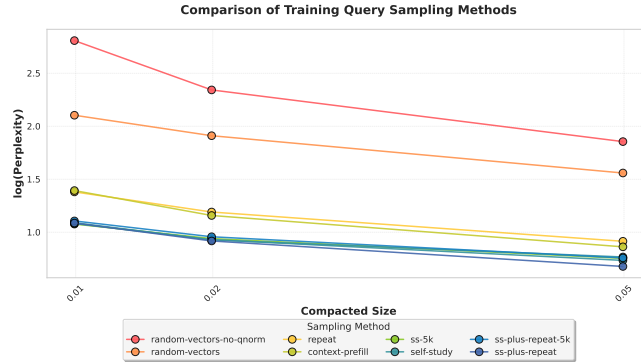


Figure 5. **Reference query sampling comparison.** We compare eight variants for sampling queries. Self-study-based methods perform best, especially at the greatest compaction ratios, with repeat and context-prefill close behind. Subsampling reference queries preserves performance.

E. Head Budgets

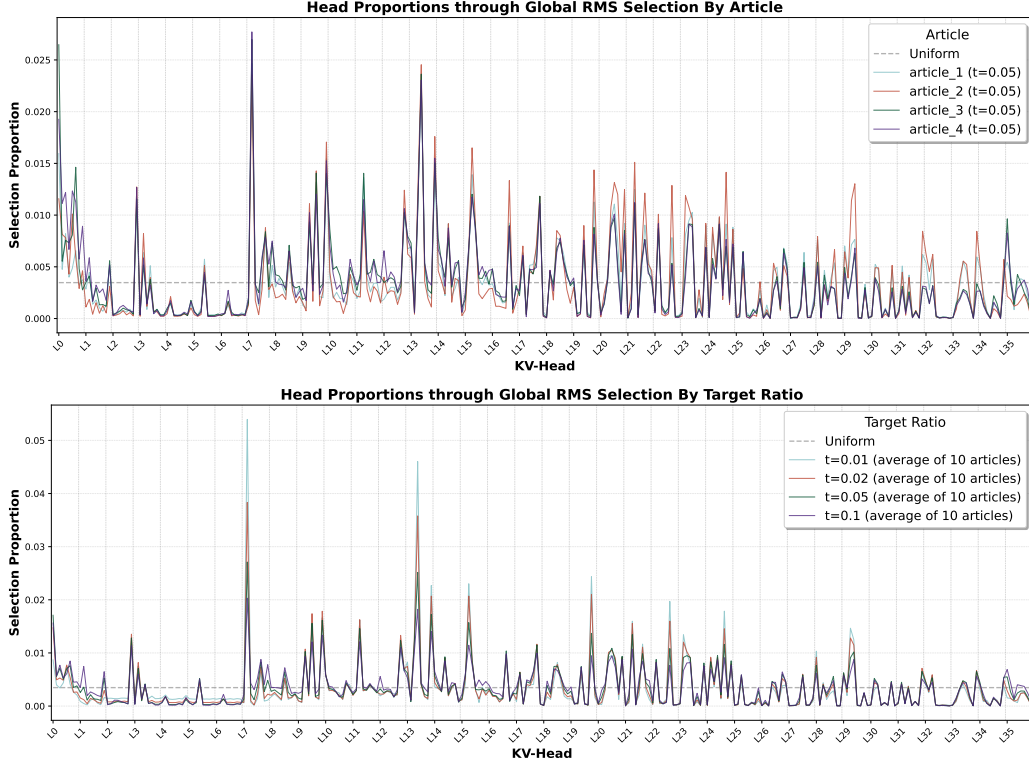


Figure 6. **Head proportions induced by global highest-attention selection.** We plot the fraction of total selected keys attributed to each of the 288 KV heads (36 layers, 8 heads per layer) in Qwen3-4B. **Top:** proportions across 10 QuALITY articles at a fixed global compaction ratio of 0.05. **Bottom:** average proportions across multiple global compaction ratios. The low variance across articles and target ratios indicates stable head importance patterns.

This appendix provides additional evidence motivating nonuniform KV compaction and visualizes the per-head budgets learned by our method.

Global key selection induces stable head proportions. KVzip (Kim et al., 2025b) selects keys based on highest attention scores *globally* across all KV heads, without enforcing per-head quotas. Under a fixed global top- k budget, this induces an implicit allocation in which different heads retain different fractions of the total KV capacity.

Figure 6 shows the implicit head proportions induced by global highest-attention selection in Qwen3-4B, supporting the use of a fixed, precomputed nonuniform schedule.

Head sensitivity curves. Following Section 3.4, we directly measure head sensitivity by varying the compaction ratio of individual heads while holding all other heads fixed. Figure 2 in the main paper shows representative sensitivity curves for Qwen3-4B.

To demonstrate that this behavior generalizes across model families, Figure 7 shows analogous head sensitivity curves for Llama-3.1-8B-Instruct. In both models, we observe substantial differences across heads: some heads are largely insensitive to KV capacity, while others benefit significantly from retaining additional keys.

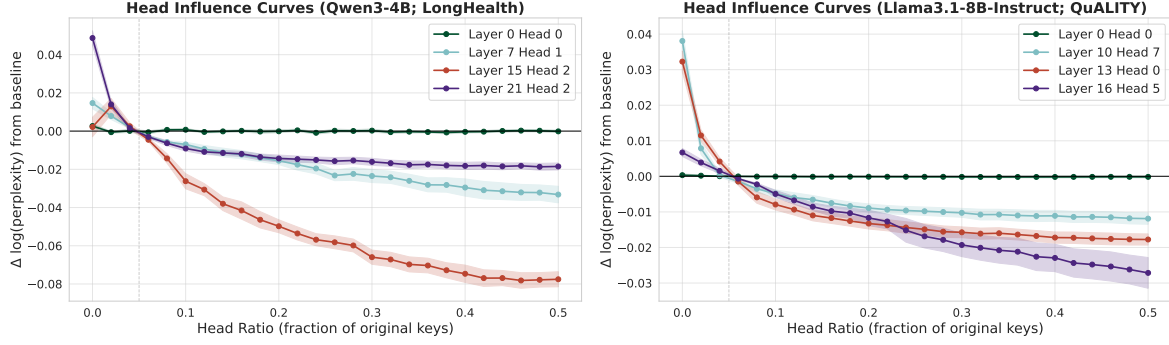


Figure 7. **More Head sensitivity curves.** Analogous to Figure 2, showing loss change as a function of per-head compaction ratio for four selected heads. **Left:** Qwen3-4B with LongHealth exhibits similar patterns to Figure 2. **Right:** Llama3.1-8B-Instruct also shows head-dependent sensitivity patterns: some heads are largely insensitive to KV capacity, while others benefit substantially from retaining more keys.

Optimized head budgets. Using these sensitivity curves, we apply the budget-allocation procedure described in Section 3.4 and Algorithm 4 to derive model-specific nonuniform head budgets. Figure 8 compares the resulting optimized head proportions to the average head proportions induced by global highest-attention key selection.

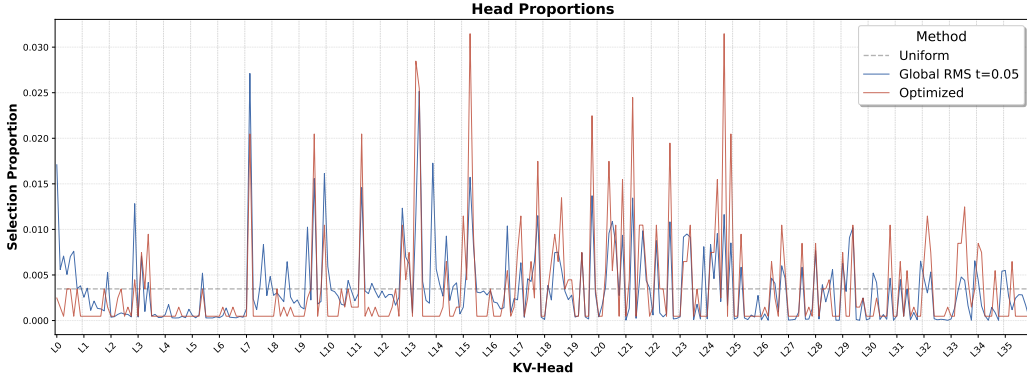


Figure 8. **Optimized head budgets.** We compare the head proportions learned by our nonuniform allocation algorithm to those induced by global highest-attention key selection. The optimized schedule has similar structure as the global highest attention scores schedule. The optimized schedule tends to assign more budget to later layers and less to earlier layers, contrary to PyramidKV (Cai et al., 2025).

Algorithm 4 Nonuniform KV Allocation via Greedy Swaps

Require: Heads $h = 1, \dots, H$; step size $\eta > 0$ (share units); per-head sensitivity curves $J_h(\rho)$ mapping compaction ratio $\rho \in [0, 1]$ to loss; reference overall compaction ratio r_0 (we use $r_0 = 0.05$).

Ensure: Shares $\{p_h\}$ with $\sum_h p_h = 1$ and $p_h \geq 0$.

- 1: **Initialize.** $p_h \leftarrow \frac{1}{H}$ for all h .
- 2: **repeat**
- 3: **Map shares to ratios.** $\rho_h \leftarrow p_h \cdot H \cdot r_0$ for all h .
- 4: **Compute marginals.** For each head h ,

$$g_h^+ \leftarrow \begin{cases} J_h(\rho_h) - J_h(\rho_h + \eta'), & \text{if } \rho_h + \eta' \leq 1, \\ -\infty, & \text{otherwise,} \end{cases} \quad g_h^- \leftarrow \begin{cases} J_h(\rho_h - \eta') - J_h(\rho_h), & \text{if } \rho_h \geq \eta', \\ +\infty, & \text{otherwise,} \end{cases}$$

where $\eta' = \eta \cdot H \cdot r_0$ is the step size in ratio space.

- 5: **Select best swap.** $b^* \leftarrow \arg \max_h g_h^+$, $a^* \leftarrow \arg \min_h g_h^-$ with $a^* \neq b^*$.
- 6: **if** $g_{b^*}^+ > g_{a^*}^-$ **then**
- 7: $p_{a^*} \leftarrow p_{a^*} - \eta$; $p_{b^*} \leftarrow p_{b^*} + \eta$.
- 8: **else**
- 9: **break** {No improving swap remains.}
- 10: **end if**
- 11: **until** termination
- 12: **return** $\{p_h\}$.

F. Further Results

F.1. Mean vs. RMS vs. Max Aggregation

Several prior methods—including H2O, SnapKV, PyramidKV, KVzip, and our *AM-HighestAttnKeys*—select a subset of keys based on attention scores under a collection of query vectors. Concretely, for each key K_j and reference query q_i , we compute the attention weight

$$a_{i,j} = \text{softmax}(q_i \mathbf{K}^\top)_j.$$

To rank keys globally, these per-query attention weights must be aggregated across queries into a single importance score per key, after which the top- t keys are retained.

We consider three aggregation functions:

$$s_j^{\text{mean}} = \frac{1}{n} \sum_{i=1}^n a_{i,j}, \quad s_j^{\text{rms}} = \sqrt{\frac{1}{n} \sum_{i=1}^n a_{i,j}^2}, \quad s_j^{\text{max}} = \max_i a_{i,j}.$$

Figure 9 compares downstream performance across these choices for four baseline methods as well as our Highest Attention Keys approach. There is no choice that is consistently best across methods, though RMS offers a good balance and we adopt it in our method.

F.2. Reconstruction vs. Downstream Accuracy

In addition to QA accuracy, we also compute loss (log-perplexity) on the *original-cache generations*: we evaluate the negative log-likelihood of responses sampled from the full cache, but conditioned on the compacted cache. This metric measures how well a compacted cache preserves the model’s token-level behavior, differing from the forward KL divergence only by a constant additive term. Figure 1 (left) replicates our main trade-off plot using this loss metric, and Figure 10 (right) compares downstream multiple-choice accuracy directly against log-perplexity across methods and compaction ratios.

Across attention-matching methods and token-selection baselines, we observe that points roughly interpolate between the *no-context* results (high loss, low accuracy) and the *full-context* results (low loss, high accuracy). As expected, improvements in reconstruction tend to translate into improvements in QA accuracy.

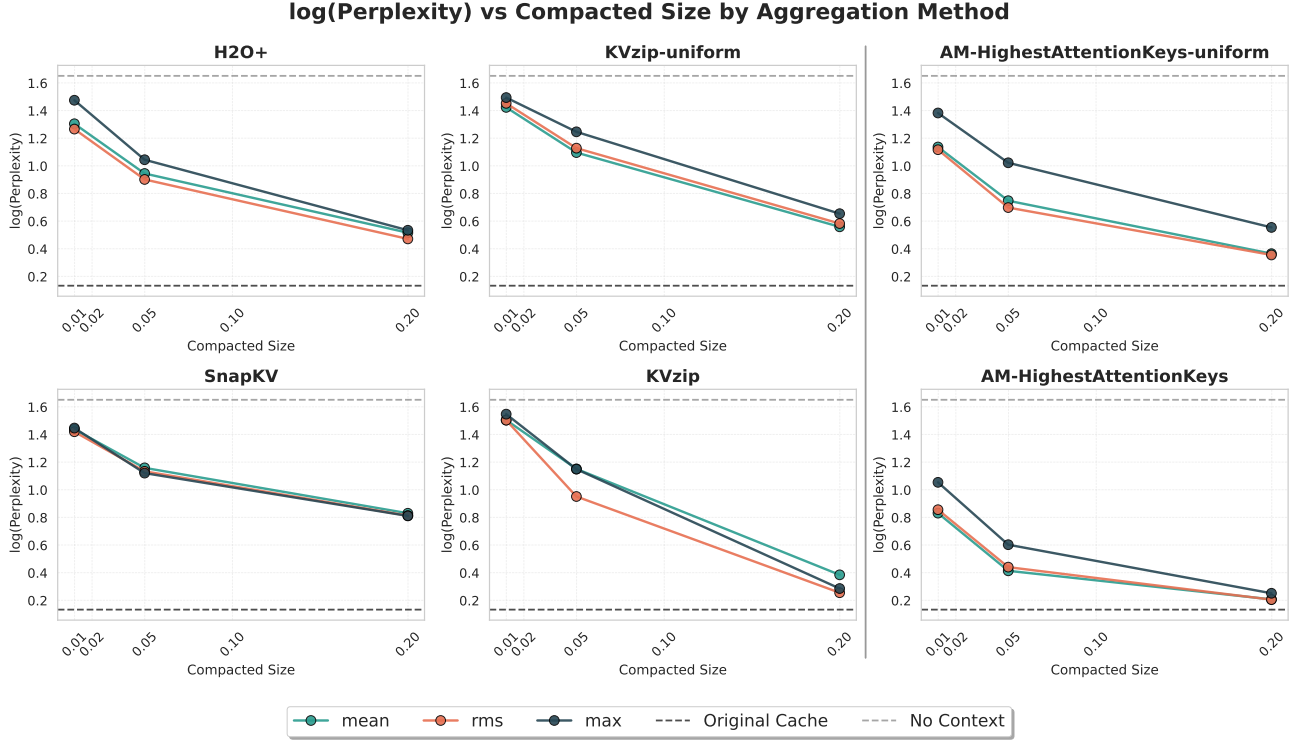


Figure 9. **Effect of aggregation function on key selection.** We plot accuracy versus compaction ratio for four baseline methods and our Highest Attention Keys method on a subset of 20 QuALITY articles using Qwen3-4B. All methods select the top- t keys based on aggregated attention scores over a shared set of reference queries.

Two notable deviations from this trend are **Cartridges** and **summarization**. Cartridges tends to achieve slightly *lower* reconstruction loss than methods with similar downstream accuracy, consistent with its objective: it performs end-to-end latent-cache optimization using a distillation-style loss that targets the original model’s behavior, directly optimizing token-level fidelity. In contrast, summarization often yields *higher* reconstruction loss than methods with similar accuracy: it discards substantial token-level detail but can preserve (or even emphasize) the subset of information most useful for answering comprehension questions. In other words, summarization can be a good task heuristic despite being a poor reconstruction mechanism.

Overall, these results support two conclusions: (i) reconstruction loss is a useful proxy for comparing compaction methods *within* families that aim to preserve attention behavior, and (ii) it should not be treated as a universal predictor of downstream performance when methods fundamentally change the objective. We further note that self-log-perplexity in long-context evaluations can be skewed on many language models due to entropy blowup as generations grow longer (Braverman et al., 2020; Cao et al., 2025). The perplexity numbers reported here are for Qwen3-4B on QuALITY (contexts < 10k tokens), where we did not observe such entropy blowup, though we did observe this at longer context lengths.

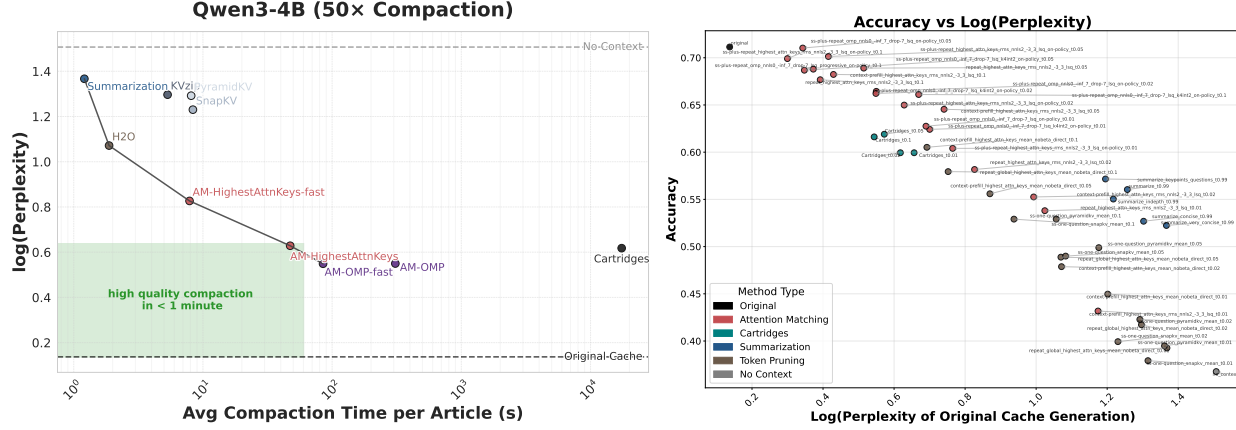


Figure 10. **Reconstruction loss vs. downstream accuracy.** **Left:** Loss-based analogue of Figure 1, instead reporting log-perplexity on original-cache generations. **Right:** Downstream multiple-choice accuracy plotted against this loss across various methods and compaction ratios.

F.3. Extension: Online Compaction

For long-horizon reasoning, we may want to compact *mid-trajectory* so the model can continue decoding after producing thousands of intermediate tokens.

Phys Len	Eff Len	AIME (/30)	Notes
2048	2048	1.25	
4096	4096	7.75	
8192	8192	13	
<i>With online compaction</i>			
2048	4096	8	≤2 compactions
2048	8192	13	≤6 compactions

Table 7. AIME performance (pass@1, averaged over 4 runs) with and without mid-trajectory compaction. *Effective sequence length* is the total number of decoded tokens, while *physical sequence length* is the maximum physical size of the KV cache. Each compaction reduces the size of the entire KV cache (including previously compacted portions) by 50%, so ≤2 and ≤6 compactions correspond to approximately 2× and 4× more decoded tokens, respectively. For implementation simplicity, we do not enable non-uniform compaction or on-policy queries. We use AM-HighestAttnKeys-Fast. We believe more tuned methods (e.g. with higher compaction ratios, compacting only the most recently generated tokens, etc.) may improve results.

We conduct a proof-of-concept evaluation of online compaction on **AIME 2025** with Qwen3-4B. For each problem, we cap the *physical* sequence length during reasoning to a fixed budget (Phys Len). Whenever the model reaches this budget, we compact the entire context *except* for the most recent 20 tokens, and then continue decoding until reaching a target *effective* decoding length (Eff Len). We compare against standard decoding to the same effective length without any compaction. To control token budgets, when the model reaches the maximum context length we inject `\nI need to respond with the answer.\n</think>Final Answer:>` (following Muennighoff et al. (2025)) and decode the final answer.

Overall, these results suggest that online compaction can effectively decouple reasoning depth from physical context limits. Despite shrinking the KV cache mid-trajectory up to six consecutive times, the model achieves performance comparable to standard decoding at the same effective length, indicating that essential reasoning state is largely preserved. Our study uses a simple uniform compaction scheme and always compacts the entire context, rather than, for example, freezing previously compacted portions or selectively compacting recent tokens. We believe that more careful tuning could enable much greater scaling. We present these results primarily to demonstrate the strong empirical stability of reasoning under repeated mid-trajectory compaction.

Finally, extending online compaction to multi-turn settings, such as compacting long tool call outputs, remains an interesting application for future work.